Formal Mathematical Definition and Equivalence Proof: The Tool Switching Problem and the Generalized Traveling Salesman Problem

Shankar Narayanan

Internship Report

Advisor: Prof. Annegret Wagler

March 30, 2026

Abstract

The management of tool configurations in Flexible Manufacturing Systems (FMS) is a well-studied combinatorial optimization challenge. Because tool switches consume valuable production time, minimizing these operations is a critical objective. This report provides a rigorous mathematical analysis of the Standard Tool Switching Problem (SSP). We explore a non-compact, configuration-based integer programming formulation that eradicates the symmetry issues plaguing traditional models. A primary contribution of this work is the formal proof of a two-way equivalence between the SSP and the Generalized Traveling Salesman Problem (GTSP). Building upon this equivalence, we propose two advanced algorithmic pathways: leveraging the Noon-Bean transformation alongside state-of-the-art routing solvers (LKH-3 and Concorde) for tractable instances, and utilizing Machine Learning-augmented dynamic graph generation to bypass the complex pricing oracle in massive industrial instances.

1 Introduction

Flexible Manufacturing Systems (FMS) enable rapid adaptation to market changes by utilizing numerically controlled machines equipped with automated tool magazines. Because a single machine may be required to process a diverse batch of parts, and because the total number of distinct tools required often exceeds the capacity of the machine's magazine, tool switches are unavoidable (**Tang1988**). A tool switch incurs a time penalty that delays production. Consequently, optimal tool management is a primary objective in operations research.

The literature broadly divides tool management into two distinct problems based on operational constraints:

1. **The Job Grouping Problem (JGP):** Occurs when a machine must be entirely stopped to change tools, allowing multiple tools to be swapped simultaneously. The objective is to partition the jobs into a minimum number of feasible groups, effectively treating the issue as a Set Covering problem (**Crama1994**).

2. **The Tool Switching Problem (SSP):** Models environments where jobs are processed sequentially and tool switches occur individually. The objective is to simultaneously determine the optimal job processing sequence and the tool loading policy to minimize the total number of individual tool switches (**Catanzaro2015**).

This report focuses exclusively on the latter: the sequential Tool Switching Problem (SSP). We aim to formalize the mathematical properties of a non-compact formulation that models the SSP as a routing problem over valid tool configurations. By rigorously proving its equivalence to the Generalized Traveling Salesman Problem (GTSP), we establish a framework capable of exploiting both decades of highly optimized routing heuristics and modern Artificial Intelligence approaches for dynamic graph generation.

2 The Non-Compact Formulation

The computational complexity of the SSP is fundamentally rooted in its state space. Traditional compact models sequence the jobs directly and track tool insertions step-by-step. However, these models suffer from extreme symmetry caused by *0-blocks*—intervals where a tool is retained in the magazine across multiple jobs despite not being required, forcing the solver to evaluate highly symmetric, redundant permutations (**Catanzaro2015**; **Laporte2004**).

To bypass this sequence-based symmetry, we utilize the non-compact formulation proposed by **Colares2026<empty citation>**, which projects the problem onto a complete directed graph of configurations.

2.1 Definitions and Variables

Let $J = \{1, 2, \ldots, n\}$ be the set of jobs and $T = \{1, 2, \ldots, m\}$ be the set of all available tools. Each job $j \in J$ requires a specific subset of tools $T_j \subseteq T$. The machine's tool magazine has a capacity C. Let $\mathcal{K} = \{K \subseteq T \mid |K| \leq C\}$ be the collection of all valid tool configurations.

We construct a complete directed graph $G = (V, E)$, where each vertex $v \in V$ corresponds to a distinct configuration $K \in \mathcal{K}$. The edge weight $d(u, v)$ represents the number of tool switches required to transition from configuration K_u to K_v:

$$d(u, v) = C - |K_u \cap K_v| \tag{1}$$

Let $H_j \subseteq V$ denote the subset of configurations capable of processing job j (i.e., $T_j \subseteq K$). The formulation utilizes two binary decision variables: $y(v) \in \{0, 1\}$ indicating if configuration v is selected, and $x(u, v) \in \{0, 1\}$ indicating a directed sequence transition between configurations u and v.

2.2 Mathematical Model

The non-compact SSP is formulated as follows:

$$\min \quad \sum_{u \in V} \sum_{v \in V} d(u, v) x(u, v) \tag{2}$$

$$\text{s.t.} \quad \sum_{v \in H_j} y(v) \geq 1 \qquad \forall j \in J \tag{3}$$

$$\sum_{u \in V} x(u, v) = y(v) \qquad \forall v \in V \tag{4}$$

$$\sum_{v \in V} x(v, u) = y(u) \qquad \forall u \in V \tag{5}$$

$$\sum_{u \in S} \sum_{v \in S} x(u, v) \leq \sum_{v \in S} y(v) - 1 \qquad \forall S \subseteq V,\ 2 \leq |S| \leq |V| - 1 \tag{6}$$

$$x(u, v) \in \{0, 1\} \qquad \forall u, v \in V \tag{7}$$

$$y(v) \in \{0, 1\} \qquad \forall v \in V \tag{8}$$

This model simultaneously solves a Covering Problem (3) and a Traveling Salesman Prob-

lem via flow conservation (4)-(5) and subtour elimination (6). While the linear relaxation is exceptionally tight, the size of V is exponential: $p = \binom{m}{C}$. Navigating this exponential graph is the primary computational challenge of the SSP.

3 Two-Way Equivalence: SSP and the GTSP

To formally bridge the Tool Switching Problem (SSP) with the Generalized Traveling Salesman Problem (GTSP), we must mathematically establish a two-way reduction. We will demonstrate that the configuration-based SSP maps to the GTSP via an exponential transformation, while the GTSP maps to the SSP via a polynomial transformation, proving that the SSP is a strictly harder generalization.

3.1 Formal Definition of Instances

We formally define an instance of the sequential SSP as $\mathcal{I}_{SSP} = \langle J, T, C, \{T_j\}_{j \in J} \rangle$, where $J = \{1, \ldots, n\}$ is the set of jobs, $T = \{1, \ldots, m\}$ is the set of tools, C is the magazine capacity, and $T_j \subseteq T$ is the tool requirement for job j. The input size of this problem is bounded by $\mathcal{O}(|J| \cdot |T|)$.

We define an instance of the GTSP using standard graph notation as $\mathcal{I}_{GTSP} = \langle V, E, d, \{V_k\}_{k \in K} \rangle$. Here, V is the set of vertices, E is the set of edges with a distance function $d : E \to \mathbb{R}^+$, and $K = \{1, \ldots, k\}$ is an index set for the clusters. The collection $\{V_k\}_{k \in K}$ is a partition of V into k mutually exclusive, disjoint clusters such that $V_p \cap V_q = \emptyset$ for all $p \neq q$ (**Pop2024**).

3.2 Reduction 1: SSP to GTSP (Forward Mapping)

In the SSP, a single tool configuration $K \in \mathcal{K}$ can contain the necessary tools to satisfy multiple jobs simultaneously. Therefore, the configuration clusters H_j are inherently overlapping ($H_i \cap H_j \neq \emptyset$). To map this to the GTSP, we adapt the Intersecting-to-Nonintersecting (I-N) transformation (**Lien1993**; **Noon1993**).

The Construction: We construct the GTSP vertex set V by duplicating valid SSP configurations into job-specific replicas:

$$V = \{v_{K,j} \mid K \in \mathcal{K},\ j \in J,\ T_j \subseteq K\} \tag{9}$$

The vertices are partitioned into exactly $|J|$ clusters. Each cluster V_j corresponds uniquely to a job $j \in J$:

$$V_j = \{v_{K,j} \mid K \in \mathcal{K},\ T_j \subseteq K\} \quad \forall j \in J \tag{10}$$

Because each replica vertex is uniquely indexed by its job j, the clusters are strictly disjoint ($V_i \cap V_j = \emptyset$). The distance function d between any two replicas in V is defined as the number of tool switches required to transition between their underlying configurations:

$$d(v_{K_1,i}, v_{K_2,j}) = C - |K_1 \cap K_2| \tag{11}$$

Proof of 0-Block Correspondence: A critical requirement of any SSP formulation is its ability to handle 0-blocks without artificial penalties. A 0-block is formally defined as a maximal interval in the job sequence where a specific tool t is not actively required, but is preceded and followed by jobs that do require it (**Catanzaro2015**; **Crama1994**). Under the optimal Keep Tool Needed Soonest (KTNS) policy, retaining tool t in the magazine during this 0-block prevents a redundant removal and re-insertion, effectively costing zero tool switches.

We must prove our transformation preserves this cost savings. Consider a 0-block spanning three consecutive jobs j_1, j_2, and j_3, where a single configuration K satisfies all three. In the SSP, holding configuration K across these jobs costs 0 switches. In our constructed GTSP, the tour must route through the respective disjoint replicas: $v_{K,j_1} \to v_{K,j_2} \to v_{K,j_3}$. The transition cost evaluated by our distance function d is:

$$Cost = d(v_{K,j_1}, v_{K,j_2}) + d(v_{K,j_2}, v_{K,j_3}) = (C - |K \cap K|) + (C - |K \cap K|) = 0 \tag{12}$$

This mathematically proves that the zero-cost edges between replicas of the same configuration allow the GTSP tour to emulate tool retention identically to the KTNS policy (**Lien1993**). Therefore, a minimum-cost GTSP tour corresponds bijectively to an optimal SSP sequence.

Computational Complexity: The number of vertices in the generated GTSP instance is $|V| \leq |J| \cdot \binom{|T|}{C}$. The reduction from the compact SSP to the GTSP is therefore **exponential** ($\mathcal{O}(|J| \cdot |T|^C)$) and not polynomially bounded.

3.3 Reduction 2: GTSP to SSP (Reverse Mapping)

To prove that the SSP is a strictly harder generalization, we must demonstrate that any arbitrary GTSP instance can be polynomially reduced to an SSP instance under a strict *Disjointness Condition*.

The Construction: Given an arbitrary GTSP instance $\mathcal{I}_{GTSP} = \langle V, E, d, \{V_k\}_{k \in K} \rangle$, we construct an SSP instance $\mathcal{I}_{SSP}$ as follows:

1. **Jobs and Capacity:** Let the set of jobs $J = K = \{1, 2, \ldots, k\}$. Set the magazine

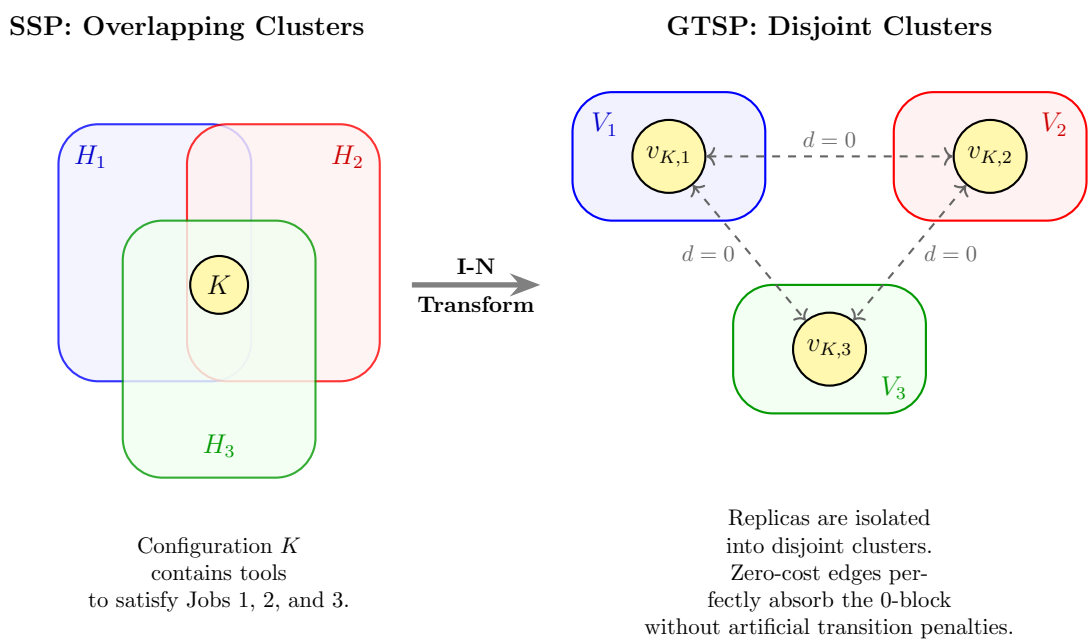

Figure 1: Forward Mapping (SSP $\rightarrow$ GTSP). A single configuration K covering three jobs is duplicated into mutually exclusive replicas connected by zero-cost edges, strictly preserving the optimal KTNS retention logic.

capacity $C = \max_{p=1}^{k} |V_p|$.

2. **Tool Padding:** Let every vertex $v \in V$ become a unique tool. To prevent any single configuration of size C from inadvertently covering multiple smaller clusters, we must enforce uniform tool requirements. For each cluster V_p, we generate a set of distinct dummy tools D_p such that $|V_p \cup D_p| = C$.

3. **Tool Set:** The complete tool repository is $T = V \cup (\bigcup_{p=1}^{k} D_p)$.

4. **Job Requirements:** For each job $p \in J$, define its tool requirement as $T_p = V_p \cup D_p$.

Proof of Correspondence: In the original GTSP, clusters are mutually exclusive ($V_p \cap V_q = \emptyset$). Because our dummy tools D_p are generated uniquely for each job, the padded tool requirements in our constructed SSP satisfy strict mutual exclusivity: $T_p \cap T_q = \emptyset$ for all $p \neq q$.

Consequently, the union of any two distinct job requirements strictly requires $2C$ tools:

$$|T_p \cup T_q| = |T_p| + |T_q| = 2C > C \quad \forall p \neq q \tag{13}$$

Under this rigorous Disjointness Condition, no valid tool configuration K (which is strictly bounded by $|K| \leq C$) can possibly contain the tools to process more than one job. This

enforces $H_p \cap H_q = \emptyset$ for all $p \neq q$. The configuration clusters in the SSP are structurally forced to be disjoint, collapsing the SSP back into a standard GTSP.

Computational Complexity: The number of jobs $|J| = k$, the capacity $C \leq |V|$, and the total number of tools $|T| \leq k \cdot |V|$. This reverse transformation is bounded by $\mathcal{O}(k \cdot |V|)$, which is strictly **polynomial**.

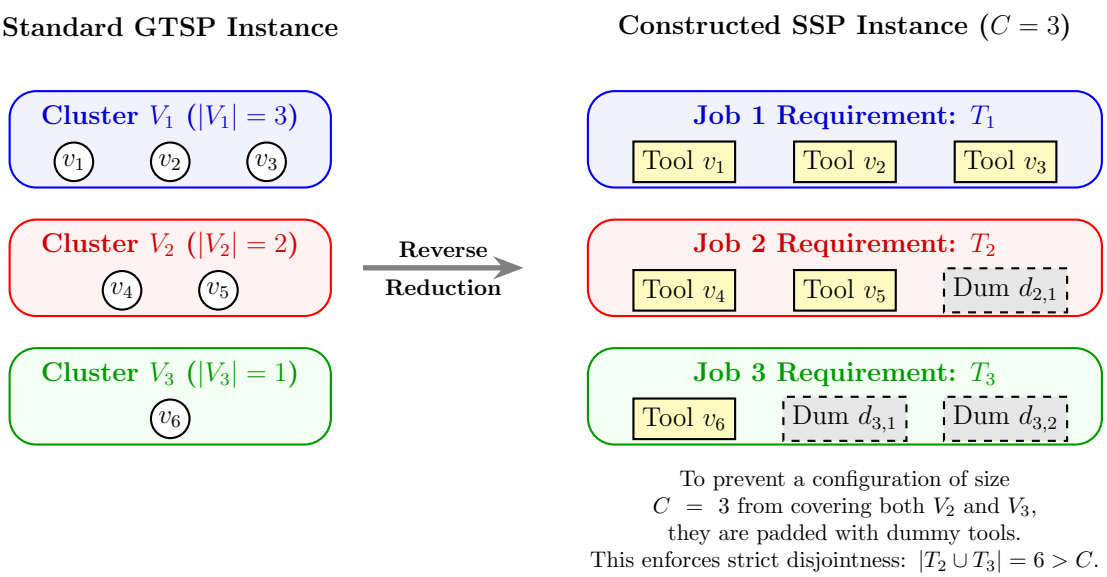

Figure 2: Reverse Mapping (GTSP $\rightarrow$ SSP). Dummy tools enforce the Disjointness Condition, ensuring that no single valid tool configuration can cover multiple jobs, collapsing the SSP back into a GTSP.

3.4 Algorithmic Implications of the Complexity Space

Because the reverse reduction is polynomial while the forward reduction is exponential, the complete configuration graph cannot always be instantiated in memory. This inherent complexity difference dictates a bifurcated solution methodology (illustrated in Figure 3).

For smaller instances where the capacity C is restrictive, the entire GTSP graph can be generated, allowing polynomial transformations (**Noon1993**) to convert the problem into an Asymmetric TSP solvable by highly optimized routing heuristics. However, for massive instances where $\binom{|T|}{C}$ triggers out-of-memory failures, exactness requires dynamic node generation via Branch-and-Price, utilizing advanced learning strategies to navigate the exponential state space.

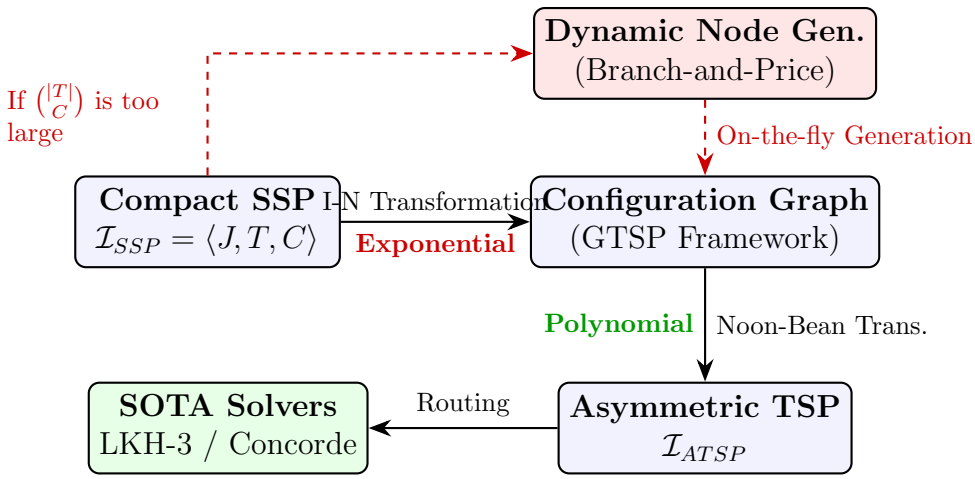

Figure 3: The complexity space of solving the SSP. Tractable instances can be reduced to ATSP and solved via highly optimized routing heuristics, while massive instances bypass the exponential bottleneck via dynamic node generation in a Branch-and-Price framework.

4 Exact Routing for Tractable Instances

For small-to-medium manufacturing environments, the total number of tools $|T|$ and the magazine capacity C restrict the configuration space $\binom{|T|}{C}$ to a computationally manageable size. Under these conditions, the theoretical equivalence established in Section 3 allows us to entirely bypass the branch-and-bound bottlenecks of generic Integer Linear Programming (ILP) solvers like CPLEX or Gurobi.

4.1 The Noon-Bean Transformation

When the configuration space is tractable, the complete GTSP graph $G = (V, E)$ and its mutually exclusive clusters $\{V_j\}_{j \in J}$ can be explicitly instantiated in memory. Once instantiated, we apply the Noon-Bean transformation (**Noon1993**). This rigorous graph transformation modifies the edge weights to compel any optimal routing algorithm to visit exactly one node per cluster before moving to the next.

Consequently, the transformation collapses the generalized problem with overlapping configurations into a standard Asymmetric Traveling Salesman Problem (ATSP) over a directed graph. The resulting ATSP perfectly encodes the 0-block tool switching mechanics without the sequence-based symmetry that paralyzes compact models.

4.2 LKH-3 and Concorde

The reduction to an ATSP unlocks the application of the most powerful routing engines developed in the operations research community. The transformed graph can be fed directly into the Lin-Kernighan-Helsgaun (LKH-3) algorithm (**Helsgaun2015**). LKH-3 utilizes highly advanced k-opt sequential exchanges and minimum spanning tree approximations to find optimal or near-optimal tours for asymmetric instances in fractions of a second (**Helsgaun2015**). For environments requiring a mathematically proven global optimum, the ATSP can be solved using exact solvers such as Concorde (**Applegate2006**).

This pathway demonstrates the immediate practical superiority of the non-compact formulation: tractable SSP instances can be translated directly into the domain of pure routing, solving in seconds what traditional compact models fail to solve in hours.

5 Dynamic Node Generation and the True Pricing Bottleneck

While the reduction to ATSP is definitive for small instances, it faces a hard mathematical limit in large-scale industrial settings. As the tool repository m and magazine capacity C grow, the number of valid configurations $\binom{m}{C}$ explodes exponentially. Generating the complete graph triggers severe out-of-memory failures. To maintain exactness without explicitly instantiating V, the problem must be solved dynamically using a Branch-and-Price (B&P) framework (**Barnhart1998**).

5.1 The Restricted Master Problem

In the B&P framework, the algorithm initializes a *Restricted Master Problem* (RMP) containing only a tiny, feasible subset of configuration nodes $V' \subset V$. The linear relaxation of the RMP is solved to obtain dual variables associated with both the job-covering constraints and the flow-conservation (routing) constraints.

To prove global optimality or find improving configurations, these dual variables are passed to a Pricing Subproblem (PSP) oracle. The oracle's task is to identify a new configuration node $v \notin V'$ that possesses a negative reduced cost, meaning its addition to the RMP will improve the objective bound.

5.2 The Complexity of the SSP Pricing Oracle

It is critical to distinguish the pricing complexity of the Tool Switching Problem (SSP) from that of the simpler Job Grouping Problem (JGP). In the JGP, the objective is purely a Set Covering task; the processing order of the configurations does not matter. Consequently, the JGP pricing oracle only needs to maximize the sum of the covering duals subject to the magazine capacity, a task formally equivalent to the Set Union Knapsack Problem (SUKP) (**Crama1994**).

However, the non-compact SSP formulation is a simultaneous covering *and sequencing* problem. Because it maps to the GTSP, the cost of a configuration heavily depends on the transition edge weights $d(u, v)$ required to travel to and from that configuration in the sequence. Therefore, the reduced cost of a new configuration node in the SSP is not just a function of its internal tools (the knapsack component); it is an explicit function of its optimal insertion point in the current routing sequence, evaluated against the routing dual variables.

To generate a new node with a negative reduced cost, the exact SSP pricing oracle must simultaneously determine the optimal subset of tools to fill the capacity C *and* compute the shortest transition paths from the existing nodes in the RMP. This creates an astronomically complex, strongly NP-hard pricing oracle. Evaluating this exact oracle at every single iteration of the column generation loop is computationally paralyzing. Thus, while the B&P architecture provides a mathematically exact framework for large instances, the complexity of the routing-based pricing oracle remains the ultimate bottleneck to scalability.

6 Machine Learning for Dynamic Node Generation

Because the exact pricing oracle must simultaneously solve a Set Covering problem for the tools and evaluate routing transitions for the sequence, executing it exactly at every iteration of the Branch-and-Price (B&P) tree is computationally paralyzing. To bridge this gap, we propose augmenting the configuration-based SSP formulation with the latest advancements in Machine Learning (ML). Rather than solving the exact oracle, ML models can be trained to heuristically predict and generate the most promising configuration nodes, falling back to the exact pricing algorithm only when necessary (**Karimi2022**).

6.1 Bipartite State Representation

To train an Artificial Intelligence agent to guide the node generation process, the operational state of the Restricted Master Problem (RMP) must be mathematically encoded. Recent

literature highlights that an RMP naturally forms a bipartite graph: one set of vertices represents the constraints (the required jobs), while the other represents the active columns (the tool configurations currently in the ATSP basis) (**Chi2022**).

By mapping the edges to indicate which configurations cover which jobs, the entire state of the SSP sequence can be processed by Graph Neural Networks (GNNs). These networks extract high-dimensional latent embeddings that capture the complex interrelations between unsatisfied jobs, current tool magazine states, and existing sequence transitions, allowing the AI to effectively "read" the optimization landscape (**Chi2022**).

6.2 Reinforcement Learning for Node Selection

Traditional Column Generation algorithms employ a myopic greedy strategy, searching for a single column with the most negative reduced cost. However, frameworks such as RLCG demonstrate that column (or node) selection is fundamentally a sequential Markov Decision Process (MDP) (**Chi2022**).

By utilizing Deep Reinforcement Learning (DRL), an agent can evaluate the bipartite GNN state and select nodes that maximize long-term rewards—specifically, minimizing the total number of pricing iterations required for the ATSP relaxation to converge. Furthermore, to combat degeneracy and accelerate routing optimization, the DRL agent can execute a *Multiple-Node Selection Strategy*. Instead of adding a single node, the agent selects a diverse, non-correlated batch of configurations to inject into the graph simultaneously, providing the routing solver with a richer subspace to optimize (**Yuan2023**).

6.3 Preserving the Exactness Guarantee

The paramount advantage of this ML-augmented pathway is that it maintains the mathematical exactness of the Branch-and-Price tree. The GNN and DRL agents act purely as a rapid heuristic filter for the Pricing Subproblem. They instantly identify and inject highly promising nodes into the ATSP graph. Only if the ML agent fails to find a configuration with a negative reduced cost does the algorithm trigger the exact pricing oracle to prove optimality (**Vaclavik2018**). Thus, the global optimality guarantee of the B&P framework remains intact, while the average computational time is drastically reduced.

7 Conclusion

The Job Sequencing and Tool Switching Problem (SSP) is a fundamental optimization challenge in flexible manufacturing. This report has demonstrated that traditional compact

integer programming formulations suffer from crippling 0-block symmetries and weak linear relaxations. By adopting a non-compact, configuration-based formulation, we mathematically eradicated these sequence-based symmetries and established a rigorous two-way equivalence between the SSP and the Generalized Traveling Salesman Problem (GTSP).

This theoretical bridge unlocks two distinct, advanced algorithmic pathways based on the complexity space of the instance. For tractable manufacturing environments, the configuration graph can be fully instantiated and mapped via the Noon-Bean transformation to an Asymmetric TSP. This allows for exact or near-exact solutions in seconds using state-of-the-art routing engines like LKH-3 and Concorde. For massive industrial instances where the graph size explodes exponentially, the formulation naturally decomposes into a Branch-and-Price framework. Because the pricing oracle must handle both tool capacities and routing transitions, it is extremely complex. However, by leveraging the bipartite structure of the Restricted Master Problem, we propose that integrating Graph Neural Networks and Deep Reinforcement Learning can successfully bypass this pricing bottleneck. Ultimately, the configuration-based formulation provides a robust mathematical foundation for hybridizing classical Operations Research with modern Artificial Intelligence, charting a definitive path toward solving the SSP at an industrial scale.